

Prompting and LLM API Notes

Message roles, structured output, context, errors, validation, and safe API handling.

Engineering notes and worked references

Contents

1. Chat messages and roles
2. Anatomy of a chat API request
3. System prompts
4. Few-shot examples
5. Temperature and variation
6. Structured JSON output
7. Context windows and token use
8. Model output is untrusted application input
9. Errors, timeouts, and retries
10. Logging model requests safely

1. Chat messages and roles

Chat-style model APIs usually accept an ordered list of messages. Each message has a role and content. The application decides which messages to send on every request.

A system message is normally used for stable application instructions. A user message carries the current request. Previous assistant and user messages can be included when the application wants the model to see earlier conversation.

Example from use

If the application sends only the current user message, the model does not automatically receive the earlier API calls.

A system instruction such as 'answer only from the supplied policy text' is different from a user asking 'what is the refund rule?'. The system message sets the operating rule, while the user message supplies the task.

Small code example

```
messages = [  
    {"role": "system", "content": "Answer in clear, short paragraphs."},  
    {"role": "user", "content": "Explain a Python for-loop."}  
]
```

Things people mix up

- Assuming the provider remembers earlier HTTP requests.
- Putting user data into a long permanent system prompt when it changes per request.
- Treating the role names as a security boundary. Application code still needs to validate input and output.

Notes

Keep stable instructions stable. If the behavior changes by product, user, or request, store that setting in normal application configuration rather than editing the prompt in many places.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For chat messages and roles, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

2. Anatomy of a chat API request

A chat request normally includes a model name, messages, and optional generation settings. The exact field names differ slightly by provider, but the basic shape is similar.

The HTTP request is only part of the flow. The application also needs to handle authentication, network timeouts, non-200 responses, malformed response bodies, and the possibility that the model returns text that does not match the requested structure.

Example from use

A request can be valid JSON and still fail because the API key is wrong or the selected model name is unavailable.

A successful HTTP response can still contain model output that your parser cannot use.

Reference table

Part	What it controls
model	Which model receives the request
messages	Instructions, context, and user input
temperature	How much variation is allowed
max tokens	Upper bound on generated output
response format	Optional structured-output behavior

Things people mix up

- Logging full authorization headers.
- Assuming HTTP 200 means the application result is valid.
- Retrying every error without checking whether the request itself is bad.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For anatomy of a chat api request, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.

- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

3. System prompts

A system prompt is useful for stable behavior such as tone, scope, output rules, or source-grounding instructions. It should be short enough that a developer can understand what the application is promising.

A long system prompt full of unrelated rules is hard to test. If a requirement can be enforced directly in code, such as validating a required JSON key or refusing an unknown session ID, code is usually the better place for it.

Example from use

A useful rule is 'If the supplied notes do not support the answer, say that the notes do not cover it.'

A weaker rule is a page of motivational wording that does not change the actual behavior.

Things people mix up

- Repeating the same instruction in system, user, and retrieved context.
- Using prompt wording as a replacement for access control.
- Putting secrets or private configuration directly in a prompt.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For system prompts, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

4. Few-shot examples

Few-shot prompting means adding examples of the input and output pattern you want. It is useful when a simple instruction leaves room for several reasonable interpretations.

Examples are especially helpful for classification, extraction, rewriting, and short structured transformations. They should demonstrate the difficult boundary cases, not only the easiest happy path.

Example from use

If a query rewriter needs to leave a new topic unchanged, include one example where the user clearly changes topic.

For a JSON extractor, include an example with a missing optional field so the desired behavior is clear.

Things people mix up

- Adding many examples that all show the same easy case.
- Using examples with fields that do not match the real application schema.
- Letting old examples silently define behavior after the product rules change.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For few-shot examples, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

5. Temperature and variation

Temperature changes how strongly the model favors its highest-probability next tokens. Lower values usually produce more consistent wording, while higher values usually allow more variation.

Temperature does not guarantee truth, determinism, or creativity. A low-temperature model can still be wrong, and two requests can still differ because of provider changes, sampling details, or model updates.

Example from use

For JSON extraction or query rewriting, lower variation is often easier to validate.

For brainstorming several lesson examples, some variation may be useful.

Things people mix up

- Treating temperature 0 as mathematically deterministic across all providers.
- Using a high temperature to fix missing knowledge.
- Changing temperature when the real problem is bad context or an unclear instruction.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For temperature and variation, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

6. Structured JSON output

Structured output is useful when downstream code needs named fields instead of a paragraph. The application should define the expected shape and then validate the returned object.

Even when a provider supports JSON mode or a schema, validation still matters. Required keys can be missing, values can have the wrong type, or the content can be logically invalid.

Example from use

A lesson generator might require title, learning_objectives, lesson_sections, practice, and quiz.

A reviewer might require issue_id, category, severity, explanation, and source.

Small code example

```
data = json.loads(model_text)

required = ["title", "learning_objectives", "quiz"]
for key in required:
    if key not in data:
        raise ValueError(f"missing required field: {key}")
```

Things people mix up

- Parsing the model response with string splitting when valid JSON is expected.
- Accepting any string in a field that should be a list or number.
- Assuming schema validity also proves factual correctness.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For structured json output, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

7. Context windows and token use

Every message sent to the model consumes context. That includes the system prompt, conversation history, retrieved passages, tool results, and the generated answer.

Longer context is not automatically better. Extra irrelevant text can make the useful evidence harder to notice and increases cost or latency. The application should send the context needed for the current task.

Example from use

A vague follow-up may need several recent messages but not the whole month-long conversation.

A RAG answer may need the top few relevant chunks rather than fifty weak matches.

Things people mix up

- Sending every conversation message forever.
- Adding large source documents directly instead of retrieving the useful sections.
- Using token limits as if they were word limits.

Quick check

- Can the idea be explained without relying on one memorized definition?

- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For context windows and token use, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

8. Model output is untrusted application input

A model response should be treated like input from any external service. It can be missing fields, contain unexpected text, request the wrong tool, or contradict the source material.

Validation belongs between the model and important application actions. This is especially important for tool arguments, database operations, filenames, and anything that changes stored state.

Example from use

A tool call requesting `top_k='five'` should be converted or rejected if the function expects an integer.

A saved note should have a normal server-generated ID instead of trusting an arbitrary identifier written by the model.

Things people mix up

- Executing model-generated code directly.
- Using model text directly in SQL.
- Assuming an apparently confident tool request is valid.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For model output is untrusted application input, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

9. Errors, timeouts, and retries

LLM calls can fail because of bad authentication, rate limits, provider outages, timeouts, invalid requests, or temporary network problems. Handle these as normal application errors.

Retries should be limited and should make sense for the error. Retrying a timeout once may be reasonable. Retrying an invalid API key five times only adds delay.

Example from use

Return a clear configuration error when the key is missing.

For a temporary rate limit, wait and retry according to the provider guidance if the application needs that behavior.

Things people mix up

- Swallowing the exception and returning an empty answer.
- Retrying forever.
- Printing the API key in the error log.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For errors, timeouts, and retries, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.

10. Logging model requests safely

Useful logs record the provider, model, request timing, request ID, response status, and enough application context to reproduce a problem. They do not need to include secrets or every piece of user content.

For debugging structured output, it can be useful to record whether parsing passed and which field failed validation. For retrieval, record the source chunk IDs rather than dumping the entire knowledge base.

Example from use

A log line such as 'model=xyz, status=200, parse=failed, missing=quiz' is more useful than 'AI failed'.

Provider request IDs are useful when support needs to correlate a failure.

Things people mix up

- Logging authorization headers.
- Logging full prompts by default when they contain private data.
- Keeping no timing or request identifier, which makes intermittent errors hard to investigate.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

A practical way to think about it

For logging model requests safely, it helps to separate what the model decides from what the application controls. The application owns the request shape, stored state, validation, retries, and any side effects. The model produces text or a structured suggestion inside those boundaries.

When something goes wrong, inspect the exact request that was sent and the response that came back before changing several prompt settings at once. A surprising answer may come from missing context, malformed JSON, an outdated system instruction, or a provider error rather than from the wording of the latest user message.

Debugging note

- Record the model name and provider.
- Record the message count and important settings.
- Keep parsing and validation errors separate from model-content errors.
- Do not log API keys or private headers.

A small reproducible request is often the fastest test. If the same minimal request works outside the full application, the problem is probably in message construction, session state, retrieval context, or response handling.